

Android 15 SystemUI 状态栏及快捷设置 客制化指导手册

文档版本
发布日期

V1.0
2024-06-24

版权所有 © 紫光展锐（上海）科技有限公司。保留一切权利。

本文件所含数据和信息都属于紫光展锐（上海）科技有限公司（以下简称紫光展锐）所有的机密信息，紫光展锐保留所有相关权利。本文件仅为信息参考之目的提供，不包含任何明示或默示的知识产权许可，也不表示有任何明示或默示的保证，包括但不限于满足任何特殊目的、不侵权或性能。当您接受这份文件时，即表示您同意本文件中内容和信息属于紫光展锐机密信息，且同意在未获得紫光展锐书面同意前，不使用或复制本文件的整体或部分，也不向任何其他方披露本文件内容。紫光展锐有权在未经事先通知的情况下，在任何时候对本文件做任何修改。紫光展锐对本文件所含数据和信息不做任何保证，在任何情况下，紫光展锐均不负任何与本文件相关的直接或间接的、任何伤害或损失。

请参照交付物中说明文档对紫光展锐交付物进行使用，任何人对紫光展锐交付物的修改、定制化或违反说明文档的指引对紫光展锐交付物进行使用造成的任何损失由其自行承担。紫光展锐交付物中的性能指标、测试结果和参数等，均为在紫光展锐内部研发和测试系统中获得的，仅供参考，若任何人需要对交付物进行商用或量产，需要结合自身的软硬件测试环境进行全面的测试和调试。

商标声明

紫光展锐、**UNISOC**、、展讯、Spreadtrum、SPRD、锐迪科、RDA 及其他紫光展锐的商标均为紫光展锐（上海）科技有限公司及/或其子公司、关联公司所有。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

免责声明

本文档可能包含第三方内容，包括但不限于第三方信息、软件、组件、数据等。紫光展锐不控制且不对第三方内容承担任何责任，包括但不限于准确性、兼容性、可靠性、可用性、合法性、适当性、性能、不侵权、更新状态等，除非本文档另有明确说明。在本文档中提及或引用任何第三方内容不代表紫光展锐对第三方内容的认可、承诺或保证。

用户有义务结合自身情况，检查上述第三方内容的可用性。若需要第三方许可，应通过合法途径获取第三方许可，除非本文档另有明确说明。

紫光展锐（上海）科技有限公司



前言

概述

本文档主要介绍 Android 15 SystemUI 的状态栏及快捷设置页面，内容涵盖状态栏和快捷设置的主要控件、主要功能、客制化方法介绍，以及常见问题分析方法。

读者对象

本文档主要适用于 Android 开发人员。

缩略语

缩略语	英文全称	中文解释
QS	Quick Settings	快捷设置

符号约定

在本文中可能出现下列符号，每种符号的说明如下。

符号	说明
 说明	用于突出重要或关键信息、补充信息和小窍门等。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害。
 注意	用于突出容易出错的操作。 “注意”不是安全警示信息，不涉及人身、设备及环境伤害。
 警告	用于可能无法恢复的失误操作。 “警告”不是危险警示信息，不涉及人身及环境伤害。

变更信息

文档版本	发布日期	修改说明
V1.0	2024-06-24	第一次正式发布

关键字

状态栏、快捷设置、SystemUI、StatusBar、QuickSettings、QS

目 录

1 状态栏	1
1.1 简介	1
1.1.1 锁屏状态栏	1
1.1.2 解锁状态栏	2
1.1.3 下拉状态栏	4
1.2 主要控件	4
1.2.1 运营商信息	4
1.2.2 时间和日期	5
1.2.3 电池	5
1.2.4 系统图标	5
1.2.5 通知图标	7
1.2.6 图标容器	9
1.3 主要功能	11
1.3.1 多图标成点	11
1.3.2 图标反色	11
1.3.3 StatusBar 服务	11
1.3.4 刘海功能	12
1.4 客制化	12
1.4.1 新增系统图标	12
1.4.2 修改系统图标	15
2 快捷设置	17
2.1 简介	17
2.1.1 QS 布局介绍	17
2.1.2 QS 界面样式	18
2.1.3 Go 版本 QS 配置	21
2.2 主要控件介绍	21
2.2.1 QSFragmentLegacy	21
2.2.2 QSPanel 和 QuickQSPanel	21
2.2.3 QSTileView	21
2.3 主要功能	22
2.3.1 QStile 点击	22
2.3.2 亮度调节	22
2.4 客制化	22
2.4.1 新增快捷设置	22
2.4.2 添加亮度模式图标	24
3 常见问题分析	25

3.1 如何去掉下拉菜单中的快捷小组件	25
3.2 如何隐藏状态栏的 VoLTE 图标	25

图目录

图 1-1 状态栏启动流程	1
图 1-2 锁屏状态栏样式	2
图 1-3 解锁状态栏样式	2
图 1-4 下拉状态栏样式	4
图 1-5 Mobile 图标更新流程	7
图 1-6 多图标成点样式	11
图 1-7 图标反色流程	11
图 2-1 下拉状态栏 QS 半展开样式	19
图 2-2 下拉状态栏 QS 全展开样式	19
图 2-3 QS 编辑界面	20

1 状态栏

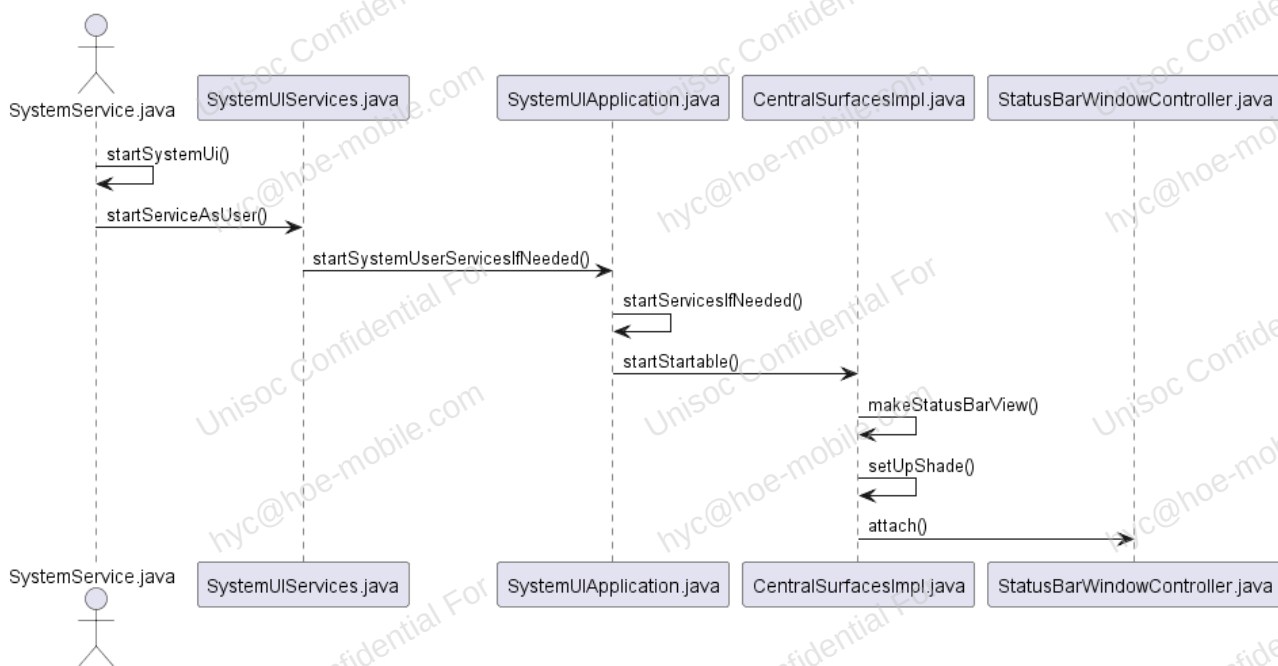
1.1 简介

状态栏（StatusBar）是 SystemUI 中的重要功能之一，主要用于显示功能图标和设备的基本信息。在 Android 15 版本中，状态栏包含以下信息：

- 功能图标主要包含 SIM 卡信息、通知图标、系统图标、Wi-Fi 图标等。
- 基本信息包含电池信息、时间、日期等。

状态栏的启动过程如图 1-1 所示。

图1-1 状态栏启动流程

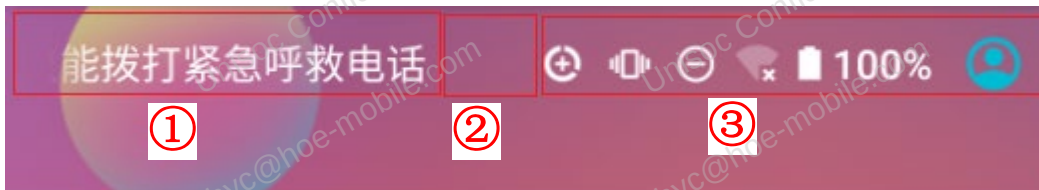


状态栏启动流程主要是在 CentralSurfacesImpl.java 中获取布局后 attach 到 window 中，其中布局的初始化在 StatusBarWindowModule.kt 中。布局名称为 super_status_bar.xml，根布局是 StatusBarWindowView，对应的子布局 status_bar_container 是 StatusBar 在解锁后的布局文件，锁屏下的布局文件为 keyguard_status_bar.xml。

1.1.1 锁屏状态栏

只有用户设置了解锁方式为滑动或者其他的安全锁时，才会显示锁屏状态栏。锁屏方式设置为无时，不会显示锁屏状态栏。只有按下 power 键或者自动息屏才会显示相应的锁屏界面。显示逻辑主要在 NotificationPanelViewController.java 中。锁屏状态下的显示效果一般如图 1-2 所示。

图1-2 锁屏状态栏样式



3 个子控件均在 `/frameworks/base/packages/SystemUI/res/layout/keyguard_status_bar.xml` 中布局，各子控件的 id 如下：

- ①: `keyguard_carrier_text`
- ②: `cutout_space_view`
- ③: `system_icons_container`

其中，`cutout_space_view` 控件大小的计算方法，请参见 `updateLayoutParamsNoCutout()`，如下所示：

```
private boolean updateLayoutParamsNoCutout() {
    if (mLayoutState == LAYOUT_NO_CUTOUT) {
        return false;
    }
    mLayoutState = LAYOUT_NO_CUTOUT;

    if (mCutoutSpace != null) {
        mCutoutSpace.setVisibility(View.GONE);
    }

    RelativeLayout.LayoutParams lp = (LayoutParams) mCarrierLabel.getLayoutParams();
    lp.addRule(RelativeLayout.START_OF, R.id.status_icon_area);

    lp = (LayoutParams) mStatusIconArea.getLayoutParams();
    lp.removeRule(RelativeLayout.RIGHT_OF);
    lp.width = LayoutParams.WRAP_CONTENT;
    lp.setMarginStart(getResources().getDimensionPixelSize(
        R.dimen.system_icons_super_container_margin_start));
    return true;
}
```

1.1.2 解锁状态栏

与锁屏后显示的状态栏不同，解锁后显示的状态栏的显示效果如图 1-3 所示。

图1-3 解锁状态栏样式



3 个子控件均在 `frameworks/base/packages/SystemUI/res/layout/status_bar.xml` 中布局，各子控件的 id 如下：

- ①: `status_bar_start_side_content`

- ②: cutout_space_view
- ③: status_bar_end_side_content

status_bar.xml 的根布局是 PhoneStatusBarView。布局代码如下所示:

- 当存在活跃通知, 且 view 的 flag 设有 SYSTEM_UI_FLAG_LOW_PROFILE 属性时会显示一个点, 提示全屏。用于当前存在通知的情况。

```
<ImageView
    android:id="@+id/notification_lights_out"
    android:layout_width="@dimen/status_bar_icon_size"
    android:layout_height="match_parent"
    android:paddingStart="@dimen/status_bar_padding_start"
    android:paddingBottom="2dip"
    android:src="@drawable/ic_sysbar_lights_out_dot_small"
    android:scaleType="center"
    android:visibility="gone"
/>
```

- 状态栏显示运营商信息。

```
<ViewStub
    android:id="@+id/operator_name_stub"
    android:layout_width="wrap_content"
    android:layout_height="match_parent"
    android:layout="@layout/operator_name" />
```

- 悬浮式通知布局。

```
<include layout="@layout/heads_up_status_bar_layout" />
```

- 时钟图标。

```
<com.android.systemui.statusbar.policy.Clock
    android:id="@+id/clock"
    android:layout_width="wrap_content"
    android:layout_height="@dimen/status_bar_system_icons_height"
    android:layout_gravity="center_vertical"
    android:textAppearance="@style/TextAppearance.StatusBar.Clock"
    android:singleLine="true"
    android:paddingStart="@dimen/status_bar_left_clock_starting_padding"
    android:paddingEnd="@dimen/status_bar_left_clock_end_padding"
    android:gravity="center_vertical|start"
/>
```

- 通知图标区域。

```
<com.android.systemui.statusbar.AlphaOptimizedFrameLayout
    android:id="@+id/notification_icon_area"
    android:layout_width="wrap_content"
    android:layout_height="match_parent"
    android:orientation="horizontal"
    android:clipChildren="false"/>
```

- 空 view, 位于状态栏中间位置。如果是刘海屏项目, 会在 updateLayoutForCutout 方法里设置区域范围。

```
<!-- Space should cover the notch (if it exists) and let other views lay out around it -->
<android.widget.Space
    android:id="@+id/cutout_space_view"
    android:layout_width="0dp"
    android:layout_height="match_parent"
    android:gravity="center_horizontal|center_vertical"
/>
```

- 系统图标区域。

```
<!-- Container that is wrapped around the views on the end half of the
status bar. Its width will change with the number of visible children and
sub-children.
It is useful when we want know the visible bounds of the content.-->
<com.android.keyguard.AlphaOptimizedLinearLayout
    android:id="@+id/status_bar_end_side_content"
    android:layout_width="wrap_content"
    android:layout_height="match_parent"
    android:layout_gravity="end"
    android:orientation="horizontal"
    android:gravity="center_vertical|end"
    android:clipChildren="false">

    <include
        android:id="@+id/user_switcher_container"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginEnd="@dimen/status_bar_user_chip_end_margin"
        layout="@layout/status_bar_user_chip_container" />

    <include layout="@layout/system_icons"
        android:layout_gravity="center_vertical"
        android:layout_width="wrap_content"
        android:layout_height="@dimen/status_bar_system_icons_height" />
</com.android.keyguard.AlphaOptimizedLinearLayout>
```

1.1.3 下拉状态栏

下拉状态栏是在用户下拉状态栏时显示的界面，显示效果如图 1-4 所示。

图1-4 下拉状态栏样式



3 个子控件均是在 `split_shade_status_bar` 中布局，各子控件的 id 如下：

- ①: clock
- ②: date
- ③: batteryRemainingIcon

其主要布局在 `frameworks/base/packages/SystemUI/res/layout/combined_qs_header.xml` 文件中。

1.2 主要控件

1.2.1 运营商信息

运营商信息控件有以下两个：

- `CarrierText`: 主要监听 `CarrierTextController`，在 `CarrierTextManager` 中进行 `updateCarrierText()`。路径为：`frameworks/base/packages/SystemUI/src/com/android/keyguard/CarrierTextController.java`

- **OperatorNameView**: 实现了 **KeyguardUpdateMonitorCallback** 接口, 用于监听 SIM 卡信息的变化并更新运营商信息。路径为:
`frameworks/base/packages/SystemUI/src/com/android/systemui/statusbar/OperatorNameViewController.java`

两者的区别是 **OperatorNameView** 只显示正常状态时的运营商信息, 而 **CarrierText** 显示所有状态下的运营商信息。

1.2.2 时间和日期

时间控件主要在 **Clock.java** 文件中实现, 具体路径如下:

`/frameworks/base/packages/SystemUI/src/com/android/systemui/statusbar/policy/Clock.java`

日期控件主要在 **DateView.java** 文件中实现, 具体路径如下:

`/frameworks/base/packages/SystemUI/src/com/android/systemui/statusbar/policy/DateView.java`

注意

日期格式不受 **Settings** 里的日期格式控制。如需修改可以添加 `systemui:datePattern` 属性, 然后自定义日期格式。

1.2.3 电池

电池控件主要实现充电动画功能, 由文件

`/frameworks/base/packages/SystemUI/src/com/android/systemui/battery/BatteryMeterView.java` 控制。

电池百分比显示功能有以下两种方式控制:

- 调用 `setForceShowPercent()` 方法来强制显示百分比, 这是锁屏状态栏显示百分比使用的方式。
- 设置 `Setting.System` 数据库里的 `SHOW_BATTERY_PERCENT` 值来控制, 这是 **Settings** 设置显示百分比的方式。

1.2.4 系统图标

1.2.4.1 定义

系统图标显示在状态栏的右侧部分, 定义在 `/frameworks/base/core/res/res/values/config.xml` 的 `config_statusBarIcons` 里, 代码如下所示:

```
<string-array name="config_statusBarIcons">
    <item><xliff:g id="id">@string/status_bar_no_calling</xliff:g></item>
    <item><xliff:g id="id">@string/status_bar_call_strength</xliff:g></item>
    <item><xliff:g id="id">@string/status_bar_alarm_clock</xliff:g></item>
    <item><xliff:g id="id">@string/status_bar_rotate</xliff:g></item>
    <item><xliff:g id="id">@string/status_bar_headset</xliff:g></item>
    <item><xliff:g id="id">@string/status_bar_data_saver</xliff:g></item>
    <item><xliff:g id="id">@string/status_bar_ime</xliff:g></item>
    <item><xliff:g id="id">@string/status_bar_sync_failing</xliff:g></item>
    <item><xliff:g id="id">@string/status_bar_sync_active</xliff:g></item>
    <item><xliff:g id="id">@string/status_bar_nfc</xliff:g></item>
    <item><xliff:g id="id">@string/status_bar_tty</xliff:g></item>
    <item><xliff:g id="id">@string/status_bar_speakerphone</xliff:g></item>
    <item><xliff:g id="id">@string/status_bar_cdma_eri</xliff:g></item>
    <item><xliff:g id="id">@string/status_bar_data_connection</xliff:g></item>
    <item><xliff:g id="id">@string/status_bar_phone_evdo_signal</xliff:g></item>
    <item><xliff:g id="id">@string/status_bar_phone_signal</xliff:g></item>
    <item><xliff:g id="id">@string/status_bar_secure</xliff:g></item>
    <item><xliff:g id="id">@string/status_bar_managed_profile</xliff:g></item>
    <item><xliff:g id="id">@string/status_bar_connected_display</xliff:g></item>
    <item><xliff:g id="id">@string/status_bar_vpn</xliff:g></item>
    <item><xliff:g id="id">@string/status_bar_bluetooth</xliff:g></item>
    <item><xliff:g id="id">@string/status_bar_camera</xliff:g></item>
    <item><xliff:g id="id">@string/status_bar_microphone</xliff:g></item>
    <item><xliff:g id="id">@string/status_bar_location</xliff:g></item>
    <item><xliff:g id="id">@string/status_bar_mute</xliff:g></item>
    <item><xliff:g id="id">@string/status_bar_volume</xliff:g></item>
    <item><xliff:g id="id">@string/status_bar_zen</xliff:g></item>
    <item><xliff:g id="id">@string/status_bar_screen_record</xliff:g></item>
    <item><xliff:g id="id">@string/status_bar_cast</xliff:g></item>
    <item><xliff:g id="id">@string/status_bar_ethernet</xliff:g></item>
    <item><xliff:g id="id">@string/status_bar_oem_satellite</xliff:g></item>
    <item><xliff:g id="id">@string/status_bar_wifi</xliff:g></item>
    <item><xliff:g id="id">@string/status_bar_hotspot</xliff:g></item>
    <!-- Unisoc: AR.254.0155.0547.4242_HD voice icon @ { -->
    <item><xliff:g id="id">@string/status_bar_hdvoice</xliff:g></item>
    <item><xliff:g id="id">@string/status_bar_mobile</xliff:g></item>
    <item><xliff:g id="id">@string/status_bar_airplane</xliff:g></item>
    <item><xliff:g id="id">@string/status_bar_battery</xliff:g></item>
    <item><xliff:g id="id">@string/status_bar_sensors_off</xliff:g></item>
</string-array>
```

1.2.4.2 图标控件

系统图标控件分为以下三种：

- ModernStatusBarWifiView 对应 Wi-Fi 图标。
- ModernStatusBarMobileView 对应信号图标。
- StatusBarIconView 对应其他系统的图标，如蓝牙、热点、闹钟等。

系统图标的显示过程不同于其他状态栏图标。系统图标只有在某些功能发现变化时才会显示或者更新。系统图标的更新控制主要集中在 PhoneStatusBarPolicy.java 和 StatusBarSignalPolicy.java 文件中，说明如下：

- PhoneStatusBarPolicy 主要控制状态类系统图标。
- StatusBarSignalPolicy 主要控制信号类系统图标。

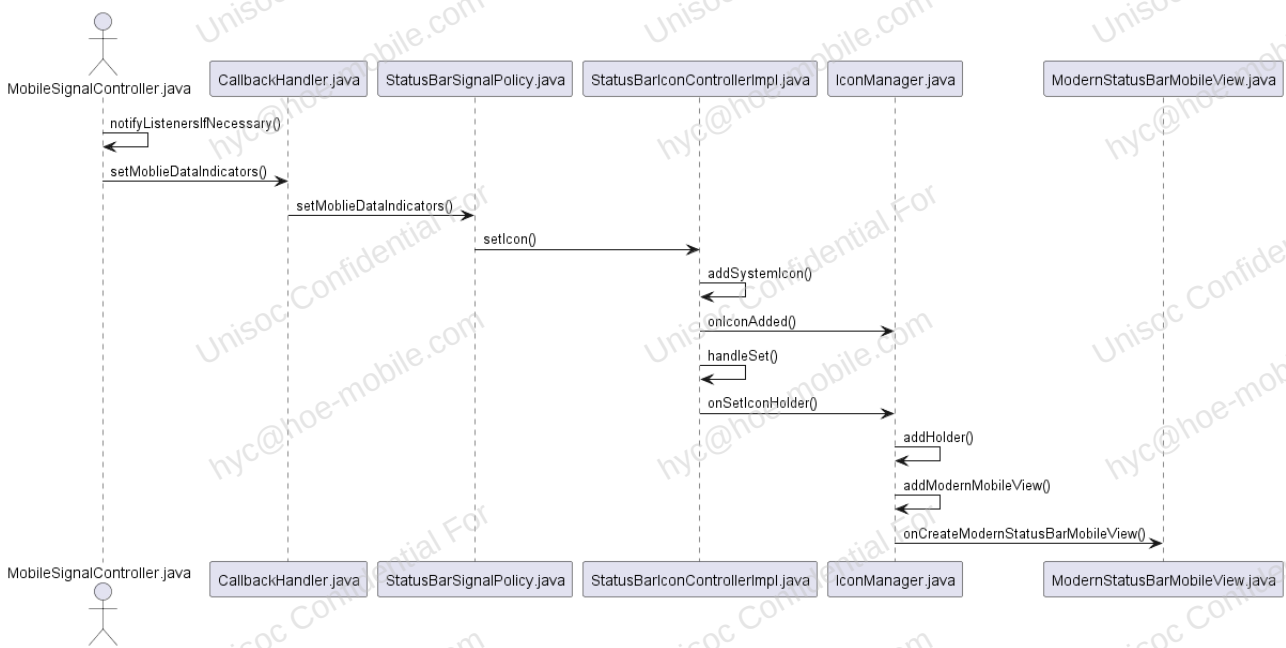
1.2.4.3 显示流程

系统图标的功能控制类不同，但显示流程基本相同，如下：

1. 通过各自的 Controller 类监听功能状态。
2. 通过 Callback 接口调用 Policy 类。
3. 再通过 IconController 来刷新图标。

以 Mobile 图标为例，更新过程如图 1-5 所示。

图1-5 Mobile 图标更新流程



1.2.5 通知图标

通知图标显示在状态栏的左侧部分，每次有通知到来时，都会在状态栏左侧显示通知所携带的通知图标。通知图标的创建与显示主要在 `IconManager.kt` 和 `StatusBarIconView.java` 中实现，在 `IconManager.kt` 中创建图标，在 `StatusBarIconView.java` 中设置图标使其在状态栏中显示。关键代码如下：

- `/frameworks/base/packages/SystemUI/src/com/android/systemui/statusbar/notification/icon/IconManager.kt`


```

/**
 * Inflate icon views for each icon variant and assign appropriate icons to them. Stores the
 * result in [NotificationEntry.getIcons].
 *
 * @throws InflationException Exception if required icons are not valid or specified
 */
@Throws(InflationException::class)
fun createIcons(entry: NotificationEntry) =
    traceSection("IconManager.createIcons") {
        // Construct the status bar icon view.
        val sbIcon = iconBuilder.createIconView(entry)
        sbIcon.scaleType = ImageView.ScaleType.CENTER_INSIDE

        // Construct the shelf icon view.
        val shelfIcon = iconBuilder.createIconView(entry)
        shelfIcon.scaleType = ImageView.ScaleType.CENTER_INSIDE
        shelfIcon.visibility = View.INVISIBLE

        // Construct the aod icon view.
        val aodIcon = iconBuilder.createIconView(entry)
        aodIcon.scaleType = ImageView.ScaleType.CENTER_INSIDE
        aodIcon.setIncreasedSize(true)

        // Set the icon views' icons
        val (normalIconDescriptor, sensitiveIconDescriptor) = getIconDescriptors(entry)

        try {
            setIcon(entry, normalIconDescriptor, sbIcon)
            setIcon(entry, sensitiveIconDescriptor, shelfIcon)
            setIcon(entry, sensitiveIconDescriptor, aodIcon)
            entry.icons = IconPack.buildPack(sbIcon, shelfIcon, aodIcon, entry.icons)
        } catch (e: InflationException) {
            entry.icons = IconPack.buildEmptyPack(entry.icons)
            throw e
        }
    }
}

```

- /frameworks/base/packages/SystemUI/src/com/android/systemui/statusbar/StatusBarIconView.java

```
/**
 * Returns whether the set succeeded.
 */
public boolean set(StatusBarIcon icon) {
    final boolean iconEquals = mIcon != null && equalIcons(mIcon.icon, icon.icon);
    final boolean levelEquals = iconEquals
        && mIcon.iconLevel == icon.iconLevel;
    final boolean visibilityEquals = mIcon != null
        && mIcon.visible == icon.visible;
    final boolean numberEquals = mIcon != null
        && mIcon.number == icon.number;
    mIcon = icon.clone();
    setContentDescription(icon.contentDescription);
    if (!iconEquals) {
        if (!updateDrawable(false /* no clear */)) return false;
        // we have to clear the grayscale tag since it may have changed
        setTag(R.id.icon_is_grayscale, null);
        // Maybe set scale based on icon height
        maybeUpdateIconScaleDimens();
    }
    if (!levelEquals) {
        setImageLevel(icon.iconLevel);
    }

    if (!numberEquals) {
        if (icon.number > 0 && getContext().getResources().getBoolean(
            R.bool.config_statusBarShowNumber)) {
            if (mNumberBackground == null) {
                mNumberBackground = getContext().getResources().getDrawable(
                    R.drawable.ic_notification_overlay);
            }
            placeNumber();
        } else {
            mNumberBackground = null;
            mNumberText = null;
        }
        invalidate();
    }
    if (!visibilityEquals) {
        setVisibility(icon.visible && !mBlocked ? VISIBLE : GONE);
    }
    return true;
}
```

1.2.6 图标容器

Android 15 中有两个容器控件，如下：

- 通知图标容器控件 NotificationIconContainer，是通知图标的父类控件。
- 系统图标容器控件 StatusIconContainer，是系统图标的父类控件。

可根据当前控件的大小来控制子控件的显示状态，状态说明如下：

- STATE_ICON: 0（表示显示图标）
- STATE_DOT: 1（表示隐藏图标但显示为圆点）
- STATE_HIDDEN: 2（表示隐藏图标）

两个容器控件的核心方法都是 calculateIconTranslations。以 StatusIconContainer 为例，说明如下：

1. 按从后到前的顺序计算当前需要显示的图标数，并计算剩余的空间 translationX。

```

/**
 * Layout is happening from end -> start
 */
private void calculateIconTranslations() {
    mLayoutStates.clear();
    float width = getWidth(); //控件宽度
    float translationX = width - getPaddingEnd(); //除去paddingEnd的宽度
    float contentStart = getPaddingStart();
    int childCount = getChildCount();
    // Underflow === don't show content until that index
    if (DEBUG) android.util.Log.d(TAG, "calculateIconTranslations: start=" + translationX
        + " width=" + width + " underflow=" + mNeedsUnderflow);

    // Collect all of the states which want to be visible
    for (int i = childCount - 1; i >= 0; i--) {
        View child = getChildAt(i);
        StatusIconDisplayable iconView = (StatusIconDisplayable) child;
        StatusIconState childState = getViewStateFromChild(child);

        if (!iconView.isIconVisible() || iconView.isIconBlocked()
            || mIgnoredSlots.contains(iconView.getSlot())) {
            childState.visibleState = STATE_HIDDEN;
            if (DEBUG) Log.d(TAG, "skipping child (" + iconView.getSlot() + ") not visible");
            continue;
        }

        // Move translationX to the spot within StatusIconContainer's layout to add the view
        // without cutting off the child view.
        translationX -= getViewTotalWidth(child);
        childState.visibleState = STATE_ICON;
        childState.xTranslation = translationX;
        mLayoutStates.add(0, childState);

        // Shift translationX over by mIconSpacing for the next view.
        translationX -= mIconSpacing;
    }

    // Show either 1-MAX_ICONS icons, or (MAX_ICONS - 1) icons + overflow
    int totalVisible = mLayoutStates.size();
    int maxVisible = totalVisible <= MAX_ICONS ? MAX_ICONS : MAX_ICONS - 1;

```

2. 通过 state.xTranslation 计算第一个无法显示的图标 index。

```

// Show either 1-MAX_ICONS icons, or (MAX_ICONS - 1) icons + overflow
int totalVisible = mLayoutStates.size();
int maxVisible = totalVisible <= MAX_ICONS ? MAX_ICONS : MAX_ICONS - 1;

mUnderflowStart = 0;
int visible = 0;
int firstUnderflowIndex = -1;
for (int i = totalVisible - 1; i >= 0; i--) {
    StatusIconState state = mLayoutStates.get(i);
    // Allow room for underflow if we found we need it in onMeasure
    if (mNeedsUnderflow && (state.xTranslation < (contentStart + mUnderflowWidth)) ||
        (mShouldRestrictIcons && visible >= maxVisible)) {
        firstUnderflowIndex = i;
        break;
    }
    mUnderflowStart = (int) Math.max(
        contentStart, state.xTranslation - mUnderflowWidth - mIconSpacing);
    visible++;
}

```

3. 通过 firstUnderflowIndex 将第一个无法显示的图标设置为 STATE_DOT，并将其前面的图标设置为 STATE_HIDDEN。

```

if (firstUnderflowIndex != -1) {
    int totalDots = 0;
    int dotWidth = mStaticDotDiameter + mDotPadding;
    int dotOffset = mUnderflowStart + mUnderflowWidth - mIconDotFrameWidth;
    for (int i = firstUnderflowIndex; i >= 0; i--) {
        StatusIconState state = mLayoutStates.get(i);
        if (totalDots < MAX_DOTS) {
            state.xTranslation = dotOffset;
            state.visibleState = STATE_DOT;
            dotOffset += dotWidth;
            totalDots++;
        } else {
            state.visibleState = STATE_HIDDEN;
        }
    }
}

```

1.3 主要功能

1.3.1 多图标成点

当系统图标区域或通知图标区域的图标过多，而区域不够时，就会将多余的图标隐藏，并显示一个点。

图1-6 多图标成点样式



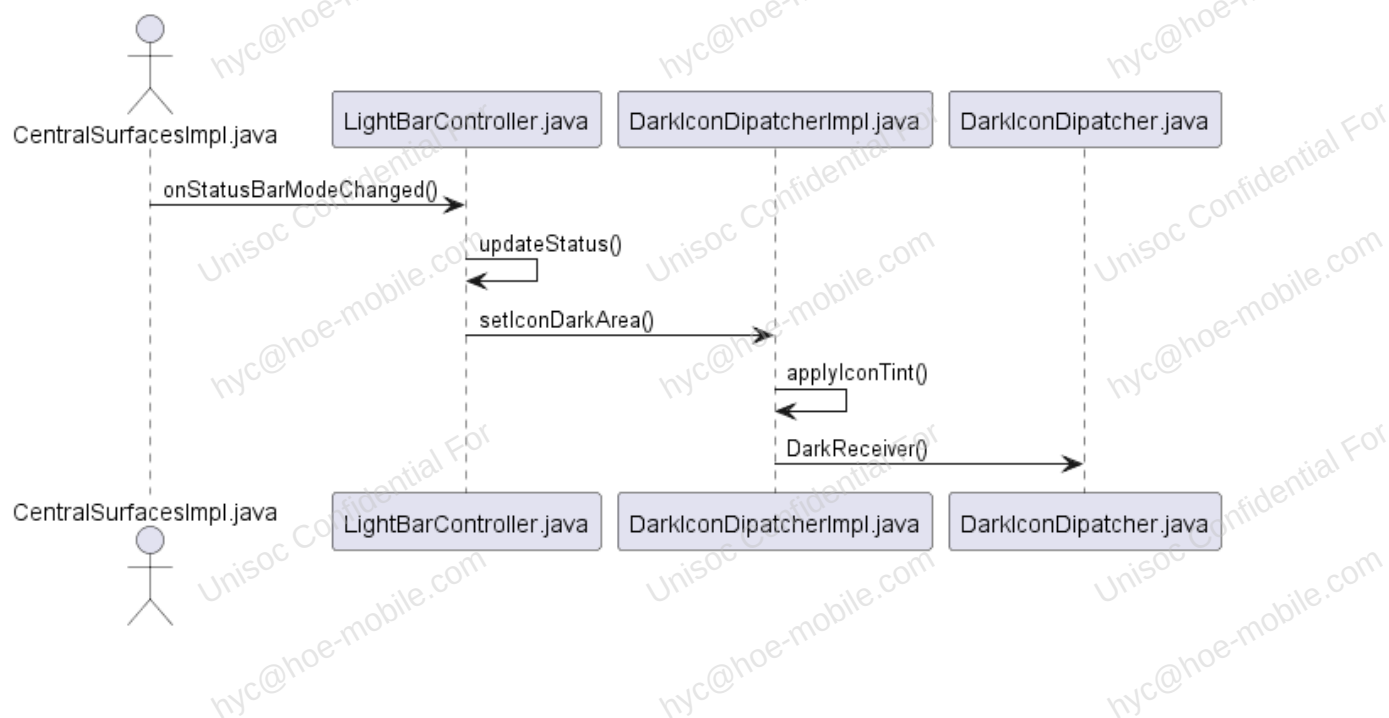
其中 `config_statusBarIcons` 参数定义了所有系统图标由低到高显示的优先级顺序。

点图标通过设置 `STATE_DOT` 来实现，在状态更新后会调用 `applyToView`，然后在图标类中调用 `setVisibleState` 来刷新新图标。

1.3.2 图标反色

当状态栏背景颜色为淡颜色时，状态栏图标会显示成黑色。背景颜色为深颜色时，图标颜色会变成白色。所有的图标类都实现了 `onDarkChanged` 方法，此方法用于设置图标颜色，具体过程如图 1-7 所示。

图1-7 图标反色流程



1.3.3 StatusBar 服务

StatusBar 服务可以实现状态栏是否显示正在使用的部分图标。StatusBar 服务是系统服务，不对外开放，只有系统应用可以直接调用（第三方应用基本上通过反射方式实现），在 `SystemService.java` 里启动。

StatusBar 服务实现了很多常用的功能，方便对状态栏进行操作，可以在 IStatusBarService.aidl 里查询定义的方法。同时 Android 也提供了 adb 命令来操作状态栏，可通过“adb shell cmd statusbar help”命令查看当前支持的 adb 功能及功能说明，如下：

```
D:\>adb shell cmd statusbar help
Status bar commands:
help
    Print this help text.

expand-notifications
    Open the notifications panel.

expand-settings
    Open the notifications panel and expand quick settings if present.

collapse
    Collapse the notifications and settings panel.

add-tile COMPONENT
    Add a TileService of the specified component

remove-tile COMPONENT
    Remove a TileService of the specified component

click-tile COMPONENT
    Click on a TileService of the specified component

check-support
    Check if this device supports QS + APIs

get-status-icons
    Print the list of status bar icons and the order they appear in
```

1.3.4 刘海功能

从 Android 9 开始新增了刘海功能，开发者模式提供了模拟刘海屏设置。在核心类 frameworks/base/core/java/android/view/DisplayCutout.java 通过配置 config_mainBuiltInDisplayCutout 值来绘制刘海区域。

在锁屏状态栏和状态栏场景下有刘海屏布局，请参考 PhoneStatusBarView.java 和 KeyguardStatusBarView.java 代码。其中，在 PhoneStatusBarView.java 中，在 updateCutoutLocation 里对刘海布局的 mCutoutSpace 进行设置。display_cutout_margin_consumption 参数用以调整刘海的宽度，常用于刘海外围弧度较大时。此时图标可以稍微显示到刘海里而不会被遮挡，可以最大范围地使用状态栏空间。

1.4 客制化

客制化功能分为新增系统图标和修改系统图标。修改完成后建议测试一下 CTS。

1.4.1 新增系统图标

系统图标主要分为状态栏图标和信号类图标。以添加反色状态系统图标为例，步骤如下：

步骤 1 在 res/drawable/stat_sys_invert_color.xml（用户可自行创建 xml 文件）中配置 config_statusBarIcons 参数（优先级不建议配置太高），添加图片资源文件。


```

--- a/core/res/res/values/config.xml
+++ b/core/res/res/values/config.xml
@@ -27,6 +27,7 @@
<!-- Do not translate. Defines the slots for the right-hand side icons. That is to say, the
     icons in the status bar that are not notifications. -->
<string-array name="config_statusBarIcons">
+
<item><xliff:g id="id">@string/status_bar_inversion</xliff:g></item>
<item><xliff:g id="id">@string/status_bar_alarm_clock</xliff:g></item>
<item><xliff:g id="id">@string/status_bar_rotate</xliff:g></item>
<item><xliff:g id="id">@string/status_bar_headset</xliff:g></item>
@@ -60,6 +61,7 @@
<item><xliff:g id="id">@string/status_bar_battery</xliff:g></item>
</string-array>

+
<string translatable="false" name="status_bar_inversion">invert_colors</string>
<string translatable="false" name="status_bar_rotate">rotate</string>
<string translatable="false" name="status_bar_headset">headset</string>
<string translatable="false" name="status_bar_data_saver">data_saver</string>

```

步骤 2 添加 Controller 文件 ColorInversionController.java 和 ColorInversionControllerImpl.java。

```

public interface ColorInversionController extends CallbackController<Callback> {

    public interface Callback {
        void onColorInversion(boolean enable);
    }
}

public ColorInversionControllerImpl(Context context) {
    mContext = context;
    mContentResolver = mContext.getContentResolver();
    mContentResolver.registerContentObserver(
        Secure.getUriFor(Secure.ACCESSIBILITY_DISPLAY_INVERSION_ENABLED), true,
        mColorInversionModeObserver);
    onColorInversionModeChanged();
    if (DEBUG) Log.d(TAG, "new ColorInversionController()");
}

public void onColorInversionModeChanged(){
    boolean enable = Secure.getIntForUser(mContentResolver, Secure.ACCESSIBILITY_DISPLAY_INVERSION_ENABLED,
        0, ActivityManager.getCurrentUser()) != 0;
    for (ColorInversionController.Callback cb : mCallbacks) {
        cb.onColorInversionChanged(enable);
    }
}

private ContentObserver mColorInversionModeObserver = new ContentObserver(new Handler()) {
    @Override
    public void onChange(boolean selfChange) { onColorInversionModeChanged(); }
}

```

步骤 3 在 Policy 里添加相关图标刷新的逻辑。本例中反色属于状态类图标，所以加到 PhoneStatusBarpolicy.java 里。


```
--- a/packages/SystemUI/src/com/android/systemui/Dependency.java
+++ b/packages/SystemUI/src/com/android/systemui/Dependency.java
@@ -65,6 +65,8 @@ import com.android.systemui.statusbar.policy.BluetoothController;
import com.android.systemui.statusbar.policy.BluetoothControllerImpl;
import com.android.systemui.statusbar.policy.CastController;
import com.android.systemui.statusbar.policy.CastControllerImpl;
+import com.android.systemui.statusbar.policy.ColorInversionController;
+import com.android.systemui.statusbar.policy.ColorInversionControllerImpl;
import com.android.systemui.statusbar.policy.ConfigurationController;
import com.android.systemui.statusbar.policy.DarkIconDispatcher;
import com.android.systemui.statusbar.policy.DataSaverController;
@@ -179,6 +181,9 @@ public class Dependency extends SystemUI {
    mProviders.put(LocationController.class, () ->
        new LocationControllerImpl(mContext, getDependency(BG_LOOPER)));

+    mProviders.put(ColorInversionController.class, () ->
+        new ColorInversionControllerImpl(mContext));
+
    mProviders.put(RotationLockController.class, () ->
        new RotationLockControllerImpl(mContext));
```

----结束

1.4.2 修改系统图标

图标有自动反色的功能，不论提供的图标资源是彩色还是黑白，都会被强制转变颜色。如果保持图标的原有颜色，需要进行相应的修改。以客制化飞行图标为例，修改步骤如下：

步骤 1 修改飞行模式图标资源

/frameworks/base/packages/SystemUI/res/drawable/stat_sys_airplane_mode.xml。

步骤 2 修改对应的视图文件。本例中飞行图标模式是 StatusBarIconView。

```
--- a/packages/SystemUI/src/com/android/systemui/statusbar/StatusBarIconView.java
+++ b/packages/SystemUI/src/com/android/systemui/statusbar/StatusBarIconView.java
@@ -889,6 +889,10 @@ public class StatusBarIconView extends AnimatedImageView implements StatusIconDi
    @Override
    public void onDarkChanged(Rect area, float darkIntensity, int tint) {
+        if("airplane".equals(mSlot)){
+            Log.i(TAG, "Flip airplane");
+            return;
+        }
        int areaTint = getTint(area, this, tint);
        ColorStateList color = ColorStateList.valueOf(areaTint);
        setImageTintList(color);
```

步骤 3 由于下拉状态栏和锁屏状态栏对 statusIcons 做了特殊处理，将其添加到 TintedIconManager 时需要修改以下文件。

```

--- a/packages/SystemUI/src/com/android/systemui/statusbar/phone/StatusBarIconController.java
+++ b/packages/SystemUI/src/com/android/systemui/statusbar/phone/StatusBarIconController.java
@@ -173,6 +173,9 @@ public interface StatusBarIconController {
    protected void onIconAdded(int index, String slot, boolean blocked,
        StatusBarIconHolder holder) {
        StatusBarIconDisplayable view = addHolder(index, slot, blocked, holder);
+       if ("airplane".equals(slot)) {
+           return;
+       }
        view.setStaticDrawableColor(mColor);
        view.setDecorColor(mColor);

@@ -183,6 +186,9 @@ public interface StatusBarIconController {
    View child = mGroup.getChildAt(i);
    if (child instanceof StatusIconDisplayable) {
        StatusIconDisplayable icon = (StatusIconDisplayable) child;
+       if ("airplane".equals(icon.getSlot())) {
+           return;
+       }
        icon.setStaticDrawableColor(mColor);
        icon.setDecorColor(mColor);
    }

```

----结束

2 快捷设置

2.1 简介

2.1.1 QS 布局介绍

QS (Quick Settings, 快捷设置) 功能是 SystemUI 的另一个重要功能, 主要提供常用功能的设置, 比如 Wi-Fi、蓝牙的开关等。用户在任意界面下拉状态栏就可以调出 QS 界面。QS 在 NotificationPanelView 里, 其布局是 status_bar_expanded.xml 里的 qs_frame。

QS 的构造在 QSTileHost 中进行。QS 的显示主要分为默认界面和编辑界面。QS 的整个布局在 frameworks/base/packages/SystemUI/res/layout/qs_panel.xml 文件中。

```
<com.android.systemui.qs.QSContainerImpl xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/quick_settings_container"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:clipToPadding="false"
    android:clipChildren="false">

    <com.android.systemui.qs.NonInterceptingScrollView
        android:id="@+id/expanded_qs_scroll_view"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:elevation="@dimen/qs_panel_elevation"
        android:importantForAccessibility="no"
        android:scrollbars="none"
        android:clipChildren="false"
        android:clipToPadding="false"
        android:layout_weight="1">

        <com.android.systemui.qs.QSPanel
            android:id="@+id/quick_settings_panel"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:background="@android:color/transparent"
            android:focusable="false"
            android:importantForAccessibility="yes"
            android:clipToPadding="false"
            android:clipChildren="false">

                <include layout="@layout/qs_footer_impl" />
            </com.android.systemui.qs.QSPanel>
        </com.android.systemui.qs.NonInterceptingScrollView>

        <include layout="@layout/quick_status_bar_expanded_header" />

        <androidx.compose.ui.platform.ComposeView
            android:id="@+id/qs_footer_actions"
            android:layout_height="@dimen/footer_actions_height"
            android:layout_width="match_parent"
            android:layout_gravity="bottom"
            android:elevation="@dimen/qs_panel_elevation"/>

        <include
            android:id="@+id/qs_customize"
            layout="@layout/qs_customize_panel"
            android:visibility="gone" />

    </com.android.systemui.qs.QSContainerImpl>
```

QS 快捷设置区域

顶部黑色状态栏

底部 action

客制化

2.1.2 QS 界面样式

2.1.2.1 默认界面

默认界面有两种形态，分别是半展开状态和完全展开状态。

- 半展开状态如图 2-1 所示。主要布局在
/frameworks/base/packages/SystemUI/res/layout/quick_status_bar_expanded_header.xml 文件中。

图2-1 下拉状态栏 QS 半展开样式



- 完全展开状态的显示效果如图 2-2 所示，包含以下两项：
 - 亮度调节显示 SeekBar（此处的亮度调节仅显示功能），对应的布局文件是 `/frameworks/base/packages/SystemUI/res/layout/quick_settings_brightness_dialog.xml`。
 - QS 的 viewPage，对应的布局文件是 `/frameworks/base/packages/SystemUI/res/layout/qs_paged_tile_layout.xml`。
- 这两个布局都是通过动态 add 添加的，不是在 `qs_panel.xml` 里直接引用。

图2-2 下拉状态栏 QS 全展开样式



2.1.2.2 编辑界面

通过点击 qs_footer_impl 里的 edit 按钮可以进入 QS 的编辑界面，edit 点击事件的处理在 QSFooterViewController.java 文件中，如下所示：

```
mEditButton.setOnClickListener(view -> {
    if (mFalsingManager.isFalseTap(FalsingManager.LOW_PENALTY)) {
        return;
    }
    mActivityIndicator
        .postQSRunnableDismissingKeyguard(() -> mQsPanelController.showEdit(view));
});
mQsPanelController.setFooterPageIndicator(mPageIndicator);
mView.updateEverything();
```

- postQSRunnableDismissingKeyguard: 先解锁，如果是安全锁需要输入安全密码解锁。
- showEdit(view): 显示客制化 QS 界面。

编辑界面显示效果如图 2-3 所示。

图2-3 QS 编辑界面



编辑界面的实现在 QSCustomizer.java 文件中，主要通过 RecyclerView 来显示各个数据，分为两个区域：

- ①：当前默认显示的 QS tile。
- ②：未默认显示的 QS tile，可以拖动到上方的默认显示区域，使其能够在 QS 列表默认显示。

2.1.3 Go 版本 QS 配置

Android 15 延用 Android 14 Go 版本使用的 Google 推荐的默认 QS 配置和 GTS 要求配置，Go 项目默认仅显示“互联网、蓝牙、飞行模式、二维码扫描器、附近分享”这 5 个 QS 按钮。如需使用其他的 QS 按钮，可以进行编辑添加其他的 QS 按钮。

Google 推荐 Go 版本配置链接如下：

<https://docs.partner.android.com/gms/building/go/config-guide-14>

展锐相关代码修改如下：

/vendor/partner_gms/unisoc_conf/overlay/gms_go_overlay_unisoc/frameworks/base/packages/SystemUI/res/values/config.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <!-- The default tiles to display in QuickSettings -->
    <string name="quick_settings_tiles_default" translatable="false">
        internet, bt, airplane, qr_code_scanner, custom(com.google.android.gms/.nearby.sharing.SharingTileService)
    </string>

    <!-- The minimum number of tiles to display in QuickSettings -->
    <integer name="quick_settings_min_num_tiles">3</integer>

    <!-- UNISOC:bug2219732 add the entries for config_quickSettingsAutoAdd start -->
    <string-array name="config_quickSettingsAutoAdd" translatable="false">
        <item>accessibility_display_inversion_enabled:inversion</item>
        <item>wind_down_first_time_setup: custom(com.google.android.apps.wellbeing/.screen.ui.GrayscaleTileService)</item>
        <item>focus_mode_first_time_setup: custom(com.google.android.apps.wellbeing/.focusmode.quicksettings.FocusModeTileService)</item>
    </string-array>
    <!-- UNISOC:bug2219732 add the entries for config_quickSettingsAutoAdd end -->
</resources>
```

2.2 主要控件介绍

2.2.1 QSFragmentLegacy

Android 15 上移除了 QSFragment 类，新增了 QsFragmentLegacy 类，此控件是整个 QS 的最大视图，主要用于控制各个 QS 界面的动画效果以及场景显示的切换。

2.2.2 QSPanel 和 QuickQSPanel

QSPanel 和 QuickQSPanel 都是 QS 的设置区域控件，主要控制 tile 的显示。QuickQSPanel 继承自 QSPanel，相对于 QSPanel，QuickQSPanel 对显示个数做了限制（TUNER_MAX_TILES_FALLBACK=6）。

2.2.3 QSTileView

此控件是 QS 里最重要的控件之一，继承自 LinearLayout，是 QS 界面控制开关的视图，定义了获取 QSTile 图标的方法 getIcon() 和获取标题的方法 getLabel()，可通过调用其子类 QSTileViewImpl 的 handleStateChanged() 方法来更新视图的状态。

2.3 主要功能

2.3.1 QStile 点击

每一个快捷设置按钮都是一个 QStileView，因此 Tile 的点击事件都是在 QStileView 里监听的，其实现主要在 `/frameworks/base/packages/SystemUI/src/com/android/systemui/qs/tileimpl/QStileViewImpl.kt` 文件中。

2.3.2 亮度调节

亮度调节由两个相同的 View 一起实现，这样在亮度调节时有更好的预览效果。

下拉状态栏完全展开时，显示的是 QSPanel 中的 View (id:brightness_slider)。

亮度调节过程如下：

1. 在滑动调节亮度时，整个下拉状态栏的透明度会被设置为 0。
2. 将 `super_notification_shade` 里的 `brightness_mirror` 设为可见。
3. 亮度调节结束后，再将下拉状态栏透明度设置为 255，隐藏 `brightness_mirror`。

下拉状态栏亮度调节的主要布局文件是

`/frameworks/base/packages/SystemUI/res/layout/quick_settings_brightness_dialog.xml`。

进度条布局文件是

`/frameworks/base/packages/SystemUI/src/com/android/systemui/settings/brightness/ToggleSeekBar.java`。

2.4 客制化

快捷设置的客制化分为新增快捷设置和添加亮度模式图标。

2.4.1 新增快捷设置

新增快捷设置有两种实现方式：

- 在 SystemUI 里实现。适用于功能要求比较复杂的场景。
- 在第三方应用里实现。适用于功能相对简单的场景。

2.4.1.1 在 SystemUI 里实现

在 SystemUI 里新增快捷设置的步骤如下：

步骤 1 新增 `*Tile.java`、`*Controller.java`、`*ControllerImpl.java` 以及所需的资源文件。

`*Tile.java` 继承 `QStileImpl<TState>`，`*Controller.java` 继承 `CallbackController<Callback>`，`*ControllerImpl.java` 实现 `*Controller.java`。

步骤 2 在 `QSFactoryImpl.java` 的 `createTile` 方法上添加 `new *Tile()`。

步骤 3 在 `frameworks\base\packages\SystemUI\res\values\config.xml` 的 `quick_settings_tiles_stock` 里添加功能字段。

如需默认显示，需要同时在 `quick_settings_tiles_default` 里添加功能字段。

----结束

2.4.1.2 在第三方应用里实现

Android 7 版本之后，Google 提供了 API 用于第三方应用添加 tile。但此 tile 只能添加在编辑界面里，由用户选择是否添加到下拉界面。

在第三方应用里新增快捷设置的步骤如下：

步骤 1 继承 android.service.quicksettings.TileService，在 AndroidManifest.xml 添加 service，如下：

```
<service
    android:name=".TestTile"
    android:label="@string/test"
    android:icon="@drawable/tile_icon_test"
    android:permission="android.permission.BIND_QUICK_SETTINGS_TILE"
    android:enabled="true">
    <intent-filter>
        <action android:name="android.service.quicksettings.action.QS_TILE" />
    </intent-filter>
</service>
```

说明

- 必须添加 permission 和 filter。
- label 是下拉状态栏显示的名称。icon 是下拉状态栏使用的图标。
- 在状态栏编辑界面，会在 label 下面显示应用名称。

步骤 2 在 java 代码的 onclick 里实现相关功能，如下：

```
package com.example.unisoc.test;

import android.service.quicksettings.TileService;
import android.util.Log;

public class TestTile extends TileService {

    private static final String TAG = "TestTile";
    private boolean mClick = false;

    @Override
    public void onStartListening() {
        super.onStartListening();
    }

    @Override
    public void onStopListening() {
        super.onStopListening();
    }

    @Override
    public void onClick() {
        super.onClick();
        mClick = !mClick;
        Log.i(TAG, msg: "mClick-->"+mClick);
    }
}
```

TileService 里没有提供 onLongClick 接口，可以长按跳转到第三方应用信息界面。

----结束

2.4.2 添加亮度模式图标

添加亮度模式图标的步骤如下：

步骤 1 在 quick_settings_brightness_dialog.xml 添加对应的 ImageView。

```
<ImageView
    android:id="@+id/brightness_icon"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_gravity="center_vertical"
    android:layout_marginEnd="8dp"
    android:src="@drawable/ic_qs_brightness_auto_off"
    android:contentDescription="@null"
    android:visibility="gone" />
```

步骤 2 在代码里根据要求将亮度模式图标显示出来。

设置图标显示的主要逻辑在 BrightnessMirrorController 中。在 showMirror 时查询当前亮度模式，并显示图标。

```
public void showMirror() {
    ImageView mIcon = mBrightnessMirror.findViewById(R.id.brightness_icon);
    mBrightnessMirror.setVisibility(View.VISIBLE);
    if (mIcon != null) {
        boolean automatic = Settings.System.getIntForUser(mContext.getContentResolver(),
            Settings.System.SCREEN_BRIGHTNESS_MODE,
            Settings.System.SCREEN_BRIGHTNESS_MODE_MANUAL,
            UserHandle.CURRENT) == Settings.System.SCREEN_BRIGHTNESS_MODE_AUTOMATIC;
        mIcon.setVisibility(View.VISIBLE);
        mIcon.setImageResource(automatic ?
            com.android.systemui.R.drawable.ic_qs_brightness_auto_on :
            com.android.systemui.R.drawable.ic_qs_brightness_auto_off);
    }
    mVisibilityCallback.accept(true);
    mNotificationPanel.setPanelAlpha(0, true /* animate */);
}
```

----结束

3 常见问题分析

3.1 如何去掉下拉菜单中的快捷小组件

默认显示的快捷小组件主要在 `/frameworks/base/packages/SystemUI/res/values/config.xml` 中的 `quick_settings_tiles_default` 里面进行配置。如需屏蔽相应的快捷小组件可在 `quick_settings_tiles_default` 修改，如下：

```
<!-- The default tiles to display in QuickSettings -->
<string name="quick_settings_tiles_default" translatable="false">
    internet,bt,flashlight,dnd,alarm,airplane,controls,wallet,rotation,battery,superbattery,cast,screenrecord,mictog
    gle,cameratoggle,custom(com.android.permissioncontroller/.permission.service.v33.SafetyCenterQsTileServi
    ce)
</string>
```

QS 半展开模式下的快捷设置项的个数，以及 QS 完全展开模式下的设置项的行列数等，如下所示：

```
<!-- The maximum number of tiles in the QuickQSPanel -->
<integer name="quick_qs_panel_max_tiles">4</integer>

<!-- The maximum number of rows in the QuickQSPanel -->
<integer name="quick_qs_panel_max_rows">2</integer>

<!-- The number of columns in the QuickSettings -->
<integer name="quick_settings_num_columns">2</integer>

<!-- The number of rows in the QuickSettings -->
<integer name="quick_settings_max_rows">4</integer>
```

3.2 如何隐藏状态栏的 VoLTE 图标

这个问题常出现在刘海屏项目中。在刘海屏项目中，状态栏图标显示区域有限，特别是当出现 VoLTE 图标时，由于 VoLTE 图标过大，导致其他的很多系统图标无法显示。

推荐方案是：在状态栏隐藏 VoLTE，在下拉状态栏显示 VoLTE。结合 `StatusIconContainer` 图标显示的逻辑，在 `calculateIconTranslations` 里判断当前空间的宽度大小。状态栏图标控件最大不会超过设备宽度的一半，而下拉状态栏大于一半。以设备宽度的 1/2 为分界点，当宽度大于分界点时允许显示 VoLTE，小于时则不显示，核心代码修改如下：


```
@@ -216,12 +217,22 @@ public class StatusIconContainer extends AlphaOptimizedLinearLayout {
    if (DEBUG) android.util.Log.d(TAG, "calculateIconTranslations: start=" + translationX
        + " width=" + width + " underflow=" + mNeedsUnderflow);

    // Collect all of the states which want to be visible
    for (int i = childCount - 1; i >= 0; i--) {
        View child = getChildAt(i);
        StatusIconDisplayable iconView = (StatusIconDisplayable) child;
        StatusIconState childState = getViewStateFromChild(child);

+         if(iconView instanceof StatusBarMobileView){
+             StatusBarMobileView mobileView = (StatusBarMobileView)iconView;
+             if(translationX < 360){ //此值可设置为手机宽度的一般修改
+                 mobileView.updateVolte(false);
+             }else{
+                 mobileView.updateVolte(true);
+             }
+         }

        if (!iconView.isIconVisible() || iconView.isIconBlocked()) {
            childState.visibleState = STATE_HIDDEN;
            if (DEBUG) Log.d(TAG, "skipping child (" + iconView.getSlot() + ") not visible");
        }
    }
}
```